1 Para qué sirve esta guía

> *Para entender el proyecto parte por parte y poder comprobar por tu cuenta si cada patrón está bien aplicado, con el código a la vista.*

La guía recorre el Sokoban tal como está hoy en el repositorio, con las cuatro partes ya unidas (arte, interfaz, dominio y sonido): qué hace cada clase, cómo se conectan y dónde está cada patrón. Cada patrón trae una lista de chequeo con el archivo y la línea donde mirar, así no hace falta creerle a nadie: abrís el archivo y lo comprobás.

La consigna deja claro por qué esto importa: **«El foco de la evaluación es la correcta aplicación de los principios y patrones de diseño presentados en la asignatura (SOLID, GRASP, MVC, GOF)»**, y además **«Cada integrante del equipo deberá realizar una defensa exhaustiva del proyecto (oral y/o escrita), lo que determinará la calificación final del mismo.»**

33	7 + 1	30 / 104	14 · 3
clases, interfaces, enums y records en uso (32 archivos .java + el record interno Foto)	patrones GoF (Observer, Facade, Singleton, Strategy, Mediator, Decorator, State) + la idea de Memento	imágenes de Martín que usa el juego	efectos de sonido (.wav) y niveles (.txt)

LA IDEA FUERZA

La vista (Luca), el dominio (Pedro y Lucía) y el sonido (Mora) no se conocen entre sí: los tres dependen de un contrato de interfaces (`Juego` , `ObservadorJuego` , `EstadoJuego`). Por eso cada parte se pudo hacer por separado y se enchufó cambiando unas pocas líneas de `Main` (y sumando cinco símbolos al dibujo del tablero).

1.1 Cómo leer este apunte

Color	Qué marca
Flúor y negrita	Texto literal de la consigna del profesor (Prof. Pepe). Es lo más importante.
Naranja	Rótulos y conceptos que ordenan el texto.
Amarillo	Definiciones.
Celeste	Explicación técnica: cómo y por qué funciona.
Verde	Lo que está bien hecho, buenas prácticas, ventajas.
Rojo	Advertencias, riesgos, cosas a mejorar.
Violeta	Ampliación tomada de la bibliografía (GoF), no de lo que hay en el proyecto.
Cian	Rótulo de ejercicio resuelto.

En las listas de chequeo: **✓ se cumple** **⚠ a medias / revisar** **× no se cumple todavía**. Las referencias tipo `vista/Imagenes.java:29` son archivo y número de línea dentro de `src/main/java/sokoban/` . En los

recuadros de código, los comentarios van en dorado: algunos ya están en el archivo y otros se agregaron para esta guía (los números de línea son los del archivo).

2 Cómo se lee un diagrama de clases

> *Lo mínimo para leer cualquier diagrama del proyecto sin perderse: la caja de cada clase, los símbolos y, sobre todo, hacia dónde apuntan las flechas.*

2.1 La caja de una clase

Tres compartimentos

Arriba el nombre; en el medio los atributos (lo que la clase guarda); abajo los métodos (lo que la clase hace). Si un compartimento está vacío igual se dibuja la rayita.

Visibilidad

`+` público (lo usa cualquiera), `-` privado (solo la clase), `#` protegido (la clase y sus hijas), `~` de paquete (sin modificador en Java: solo lo ven las clases del mismo paquete). El `~` aparece en el sonido: por ejemplo `establecerEstado` de `Sonidos`.

Subrayado y cursiva

Subrayado = `static`: pertenece a la clase y no a cada objeto (por ejemplo `obtenerInstancia()`). Cursiva = abstracto: un método sin código, como todos los de una interfaz.

Estereotipos «...»

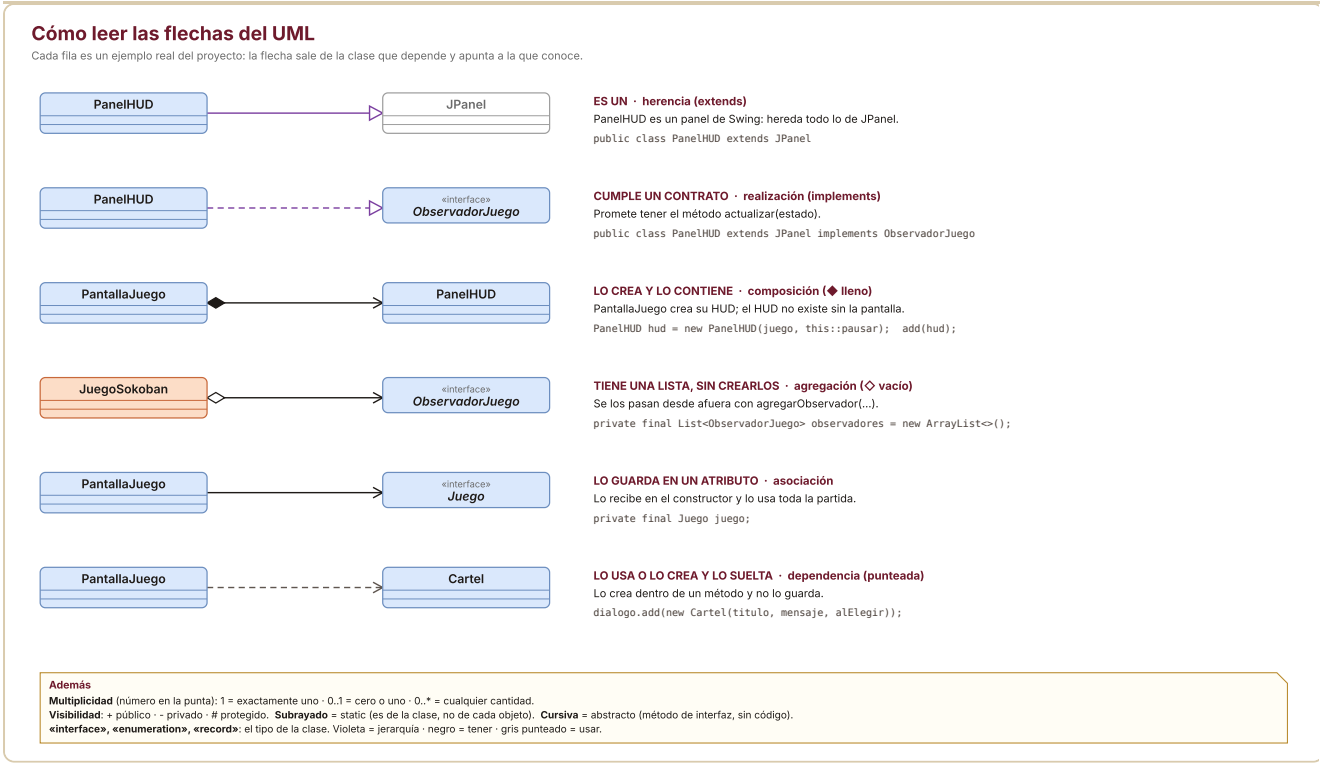
Dicen qué tipo de cosa es: «interface», «enumeration», «record». En los diagramas de patrones, la etiqueta dorada arriba de la caja dice el **rol** que cumple esa clase en el patrón.

Colores de las cajas

Autoría: azul = Luca, verde = Martín, amarillo = integración, naranja = Pedro y Lucía (dominio), lila = Mora (sonido).

2.2 Las flechas

Regla general: **la flecha sale de la clase que depende y apunta a la que conoce.** Si dudás, preguntate «¿quién tiene el código que menciona a quién?»: la flecha sale de ese archivo.



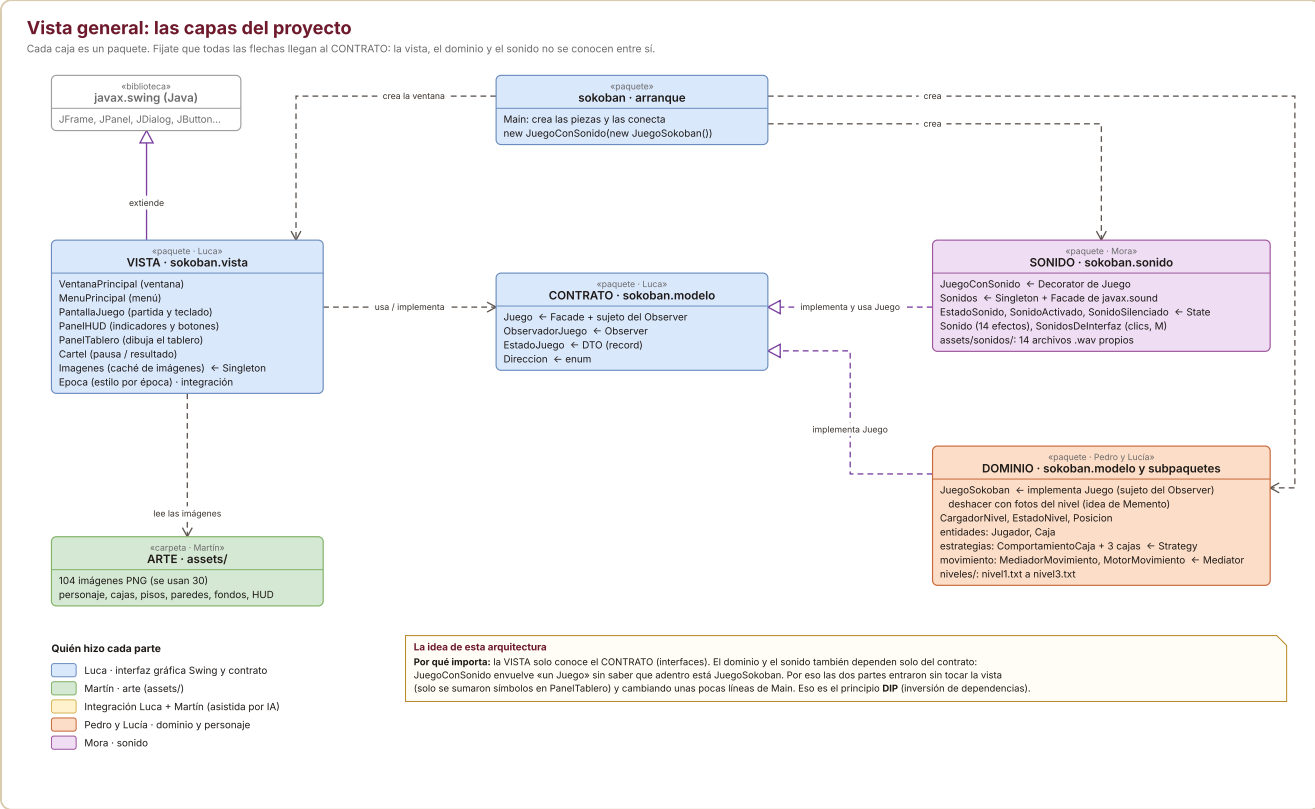
docs/uml/0-como-leer-flechas · cada fila es un ejemplo real del proyecto.

2.3 Errores típicos al leer o dibujar

- Dibujar la flecha al revés. `PantallaJuego` conoce a `Juego`, no al revés: la flecha va de `PantallaJuego` hacia `Juego`.
- Confundir herencia con realización. Herencia (`extends`) es línea llena; realización (`implements`, con una interfaz) es punteada. Las dos terminan en triángulo hueco.
- Confundir composición con agregación. Composición (◆ lleno): el todo crea a la parte y la parte no vive sin él (`EstadoNivel` crea sus `Caja`). Agregación (◇ vacío): el todo tiene a la parte pero se la dieron de afuera (los observadores de `JuegoSokoban`).
- Poner asociación donde solo hay un uso pasajero. Si la otra clase aparece solo dentro de un método (se crea o se recibe y no se guarda), es dependencia (punteada), no asociación. Ejemplo: `MotorMovimiento` mueve la `Caja` pero no la guarda.

3 El mapa del proyecto

> *Antes de entrar en las clases, la foto desde lejos: qué paquetes hay, quién hizo cada uno y quién depende de quién.*



docs/uml/1-vista-general · todas las flechas llegan al CONTRATO.

Paquete o carpeta	Qué contiene	Autor	Estado
`sokoban`	Main: arranca el programa y conecta las piezas.	Luca (+ 4 líneas de Mora)	✓ listo
`sokoban.vista`	Todo lo que se ve: ventana, menú, partida, HUD, tablero, carteles, imágenes.	Luca (+ `Epoca` en la integración y 5 símbolos nuevos del dominio)	✓ listo
`sokoban.modelo` (contrato)	El contrato entre la vista y el dominio: 2 interfaces, 1 record y 1 enum.	Luca	✓ listo
`sokoban.modelo` (dominio) y `.entidades`, `.estrategias`, `.movimiento`	Las reglas: carga de niveles, tablero, jugador, cajas normal / frágil / llave, hielo, cerrojo y muros, deshacer y puntaje.	Pedro y Lucía	✓ integrado

Paquete o carpeta	Qué contiene	Autor	Estado
`sokoban.sonido`	Efectos de sonido del juego y de la interfaz, y la tecla M para silenciar.	Mora	✓ integrado ⚠ falta aprobar el pull request
`sokoban.prueba`	JuegoDePrueba: el Sokoban mínimo que se usó para probar la vista.	Luca	⚠ ya no se usa: borrarlo
`assets/`	104 imágenes: personaje, cajas, pisos, paredes, fondos, HUD.	Martín	✓ listo (30 en uso)
`assets/sonidos/`	14 efectos .wav generados por código (`docs/generar_sonidos.py`).	Mora	✓ listo
`niveles/`	3 niveles en .txt, un carácter por celda.	Pedro y Lucía	⚠ son de prueba, muy chicos

PARA VALIDAR

Ningún archivo de `sokoban.modelo` ni de `sokoban.sonido` importa algo de `sokoban.vista`, y ni la vista ni el dominio importan algo de `sokoban.sonido`. Si algún día aparece un `import sokoban.vista…` en el dominio, se rompió la arquitectura. Comprobalo buscando «import sokoban.vista» e «import sokoban.sonido» en esos paquetes: hoy no aparece ninguno. La única clase que conoce a todos es `Main`, y está bien que así sea.

4 Qué pasa cuando jugás

> *El diagrama de clases muestra la estructura; el de secuencia muestra el orden de las llamadas. Seguir una jugada de punta a punta es la mejor forma de entender el diseño.*

4.1 El arranque

Main.java

```
17  public static void main(String[] args) {
18      SwingUtilities.invokeLater(() -> {          // todo Swing corre en un solo hilo (EDT)
19          Juego juego = new JuegoConSonido(new JuegoSokoban()); // el sonido envuelve al
dominio
20          new VentanaPrincipal(juego).setVisible(true);
21          SonidosDeInterfaz.instalar();                        // arranque, clics y tecla M
22      });
23  }
```

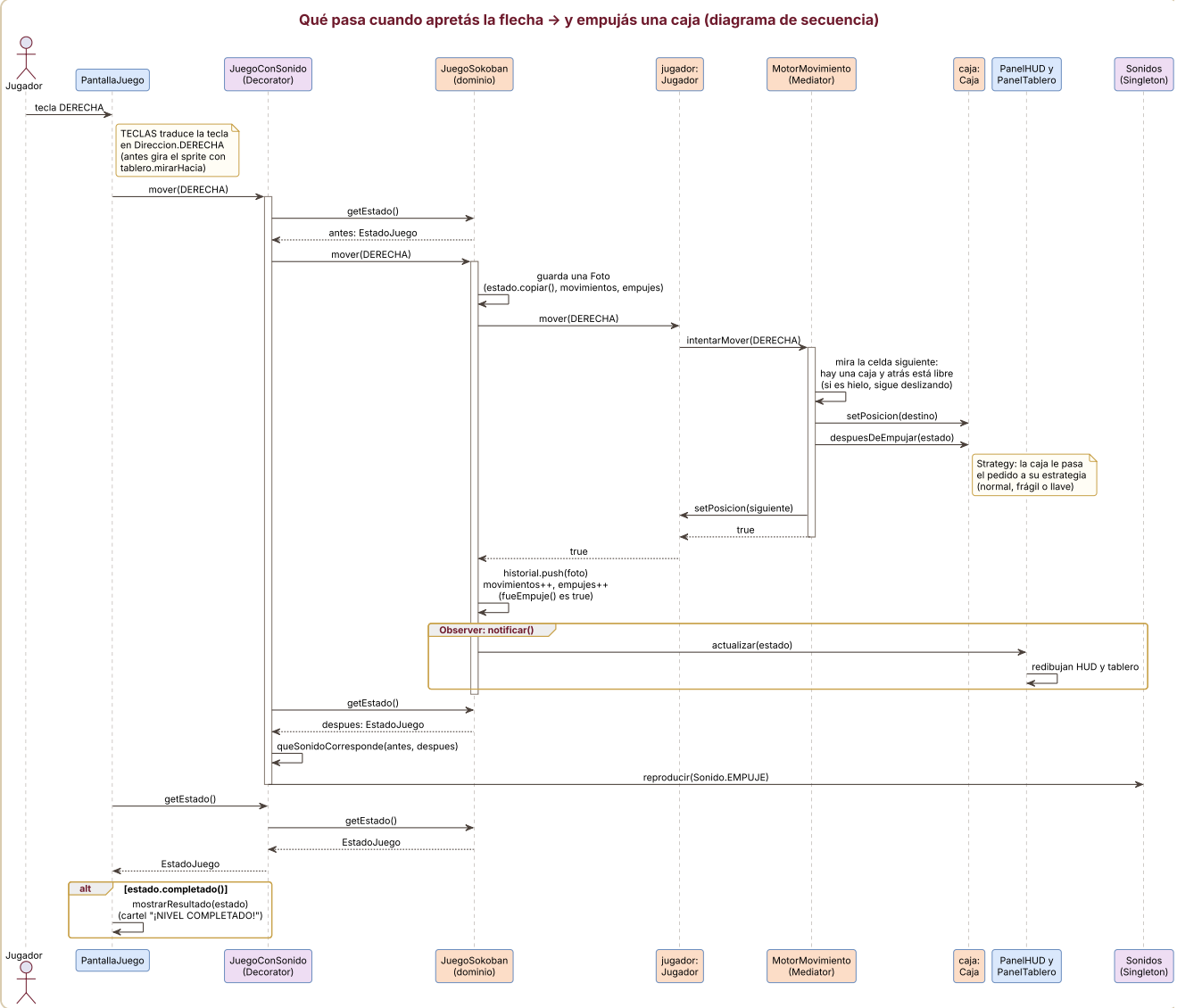
`Main` es el único lugar que nombra las clases concretas: crea el dominio (`JuegoSokoban`), lo envuelve con el sonido (`JuegoConSonido`) y le pasa el resultado a la ventana como un `Juego` (la interfaz). La ventana no sabe que adentro hay sonido ni qué clase es el dominio. `SonidosDeInterfaz.instalar()` suma el sonido de arranque, el clic de los botones y la tecla M sin tocar ninguna pantalla.

4.2 Cómo se carga un nivel

1. `PantallaJuego` pide `juego.iniciarNivel(n)`. Pasa primero por `JuegoConSonido`, que le pasa el pedido al dominio y después hace sonar el inicio de nivel.
2. `JuegoSokoban.iniciarNivel` controla que el número esté entre 1 y 3 y le pide el nivel a `CargadorNivel` (`modelo/JuegoSokoban.java:26`).
3. `CargadorNivel.cargar(n)` busca `/niveles/nivelN.txt` adentro del `.jar`; si no está, lo lee de la carpeta `niveles/` del proyecto (`modelo/CargadorNivel.java:13` y `:18`).
4. `new EstadoNivel(lineas)` recorre el archivo carácter por carácter: guarda el terreno, crea el `Jugador` y crea cada `Caja` con su estrategia (`modelo/EstadoNivel.java:34` a `:52`).
5. Valida el archivo: que no esté vacío, que todas las filas tengan el mismo ancho, que haya un solo jugador, que no haya símbolos desconocidos y que haya una caja por cada destino. Si algo está mal, avisa con un mensaje claro (`IllegalArgumentException`).
6. `JuegoSokoban` conecta el motor de movimiento al jugador (`conectarMotor`), pone los contadores en cero, vacía el historial y avisa a los observadores.

Símbolo	En el archivo .txt	Símbolo	Solo lo arma el dominio para la vista
`#` ` ` `.`	pared, piso, destino	`f`	caja frágil dañada (le queda 1 de resistencia)
`~`	piso resbaladizo (hielo)	`G` `g`	caja frágil sobre un destino (intacta / dañada)
`C` `X` `O`	cerrojo, muro cerrado, muro abierto	`K`	caja llave sobre un destino
`@` `+`	jugador (sobre piso / sobre destino)	`c`	cerrojo abierto
`$` `*` `F` `L`	caja normal (sobre piso / sobre destino), caja frágil, caja llave		Los arma `EstadoNivel.tableroParaVista()` (`:122`).

4.3 Apretar una flecha y empujar una caja, paso a paso



docs/uml/7-secuencia-mover · el color de cada participante es su autor.

1. `PantallaJuego` recibe la tecla, la traduce con el mapa `TECLAS` en `Direccion.DERECHA` y gira el sprite del jugador (`vista/PantallaJuego.java:56` y `:72`).
2. Le pide al juego `mover(DERECHA)`. Quien atiende primero es `JuegoConSonido`: anota el estado de antes y le pasa el pedido a `JuegoSokoban`.
3. `JuegoSokoban` saca una foto del nivel (para deshacer) y le pide al `Jugador` que se mueva.
4. **Mediator:** el `Jugador` no sabe de paredes ni cajas: le pasa el pedido a su mediador, `MotorMovimiento`, que revisa la celda siguiente, empuja la caja (y la deja deslizar si cae en hielo) y mueve al jugador.
5. **Strategy:** después del empuje, el motor le avisa a la `Caja` con `despuesDeEmpujar`, y la caja le pasa el aviso a su estrategia: la normal no hace nada, la frágil pierde resistencia y la llave abre los muros si quedó sobre un cerrojo.
6. Si el movimiento se pudo hacer, `JuegoSokoban` guarda la foto en el historial, suma un movimiento (y un empuje si lo hubo) y llama a `notificar()`.
7. **Observer:** `notificar()` manda el nuevo `EstadoJuego` al HUD y al tablero, y cada uno se redibuja solo.
8. **Decorator:** de vuelta en `JuegoConSonido`, compara el estado de antes con el de después y hace sonar el efecto que corresponde (acá, empuje) con el Singleton `Sonidos`.

9. `PantallaJuego` pide `getEstado()`; si `completado()` es verdadero, muestra el cartel de resultado (`vista/PantallaJuego.java:75`).

Fijate que en ningún paso la vista toca el tablero del dominio: solo pide y recibe una foto (`EstadoJuego`).

4.4 El puntaje

«El criterio utilizado para calcular dicho puntaje debe ser definido por el equipo de desarrollo.» El equipo eligió esta fórmula (`modelo/JuegoSokoban.java:80`):

```
modelo/JuegoSokoban.java
80  int puntaje = Math.max(0, 1000 - 10 * movimientos - 5 * empujes - 50 * undosUsados);
```

- Se arranca con 1000 puntos y se resta 10 por movimiento, 5 por empuje y 50 por cada deshacer usado. Nunca baja de 0.
- Ejemplo: 20 movimientos, 6 empujes y 1 deshacer dan 1000 - 200 - 30 - 50 = 720 puntos.
- Al deshacer, los movimientos y empujes vuelven a lo que decía la foto (dejan de contar), pero el deshacer suma su propia penalidad. Así deshacer sirve, pero no sale gratis.

5 Clase por clase

> *Qué hace cada clase, con quién se relaciona y qué mirar para validarla. Está ordenado por paquete, de afuera (lo que se ve) hacia adentro (las reglas).*

5.1 Vista · `sokoban.vista` (Luca)

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
VentanaPrincipal extiende JFrame	La ventana. Tiene el menú y la partida, y cambia entre ellos con un `CardLayout`.	◆ MenuPrincipal, ◆ PantallaJuego	Las pantallas no se conocen entre sí: se avisan con callbacks (`this::jugar`, `this::mostrarMenu`).
MenuPrincipal extiende JPanel	Menú con un botón por nivel y Salir, sobre el fondo de 1985.	◆ Cartel, usa Imagenes	Recibe `IntConsumer` y `Runnable`: no sabe quién lo contiene (bajo acoplamiento).
PantallaJuego extiende JPanel	La partida: HUD arriba, tablero al centro. Traduce teclas en movimientos y muestra pausa y resultado.	→ Juego, ◆ PanelHUD, ◆ PanelTablero, crea Cartel	Es la clase con más tareas (armar, teclado, carteles). Ver SRP en la sección 16.
PanelHUD extiende JPanel	Indicadores de nivel, movimientos y empujes, y botones Deshacer, Reiniciar y Pausa.	implementa ObservadorJuego, usa Juego e Imagenes	Los botones no toman el foco, así las flechas siempre mueven al jugador (`vista/PanelHUD.java:82`).

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
PanelTablero extiende JPanel	Dibuja el tablero celda por celda con las imágenes de la época, y el personaje mirando hacia donde se movió.	implementa ObservadorJuego, → Epoca, → Direccion, usa Imagenes	Los símbolos están en el mapa `CAPAS`: los 5 símbolos nuevos del dominio entraron como 5 líneas, sin `switch` (`:35`).
Cartel extiende JPanel	Recuadro reutilizable con título, mensaje y botones.	lo usan el menú, la pausa y el resultado	Evita código duplicado: los tres carteles son la misma clase.
Imagenes «Singleton»	Caché de imágenes: cada PNG se lee del disco o del .jar una sola vez.	la usan Menu, HUD y Tablero; lee assets/	Ver sección 9 (Singleton).
Epoca enum · integración	Fondo, piso y pared de cada época (1955, 1985, 2015). Los niveles las van rotando.	la usa PanelTablero	Una época nueva es una constante más (OCP). `Math.floorMod` evita índices negativos.

5.2 Contrato · `sokoban.modelo` (Luca)

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
Juego interface	Todo lo que la vista le puede pedir al dominio: iniciar nivel, mover, deshacer, reiniciar, pedir el estado y registrar observadores.	Facade, sujeto del Observer y componente del Decorator	El comentario de `modelo/Juego.java:10` fija la regla: después de cada cambio hay que avisar a los observadores.
ObservadorJuego interface	Un solo método: `actualizar(EstadoJuego)`.	la implementan PanelHUD y PanelTablero	Un método solo: cumple ISP.
EstadoJuego record (DTO)	La foto del nivel: nivel, movimientos, empujes, deshacer usados, puntaje, si ganó y el tablero como líneas de texto.	lo devuelve Juego y lo recibe cada observador	Es inmutable: `record` + `List.copyOf` (`modelo/EstadoJuego.java:16`).
Direccion enum	ARRIBA, ABAJO, IZQUIERDA, DERECHA; cada una sabe cuánto se mueve en filas y columnas.	la usan la vista, Juego, Jugador, Posicion y el mediador	Evita «primitive obsession» (no se usan números o textos sueltos) y es Experto en información (GRASP).

5.3 Dominio · `sokoban.modelo` y subpaquetes (Pedro y Lucía)

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
JuegoSokoban	Implementa `Juego`: recibe los pedidos de la vista, guarda el historial para deshacer, cuenta movimientos, empujes y deshacer, calcula el puntaje y avisa a los observadores.	implementa Juego, ◆ CargadorNivel, → EstadoNivel, → MotorMovimiento, ◇ ObservadorJuego, ◆ Foto	Es el Controlador de GRASP y la fachada concreta. Las constantes 3, 15, 5 y 3 tienen nombre (`:10` a `:13`).
Foto record privado	Una copia del nivel más los contadores de movimientos y empujes.	la guarda JuegoSokoban en el historial	Ver sección 12 (deshacer).
CargadorNivel	Lee `nivelN.txt` del .jar o de la carpeta `niveles/`.	crea EstadoNivel	Separa «leer el archivo» de «entender el nivel».
EstadoNivel	El tablero vivo: terreno, jugador y cajas. Sabe si una celda está libre, si es hielo o cerrojo, abre los muros, saca cajas rotas, sabe si se ganó, arma el texto para la vista y se copia a sí mismo.	◆ Jugador, ◆ Caja (0..*)	Valida el archivo y explica el error. Es la clase más larga del dominio (158 líneas).
Posicion record	Fila y columna; `siguiente(direccion)` da la celda de al lado.	la usan Jugador, Caja y el motor	Inmutable y comparable con `equals` (por ser record).
Jugador entidades	Tiene una posición y un mediador; para moverse le pide permiso al mediador.	→ MediadorMovimiento	Ver sección 11 (Mediator).
Caja entidades	Tiene posición, resistencia y una estrategia que decide qué pasa después de cada empuje y con qué símbolo se dibuja.	→ ComportamientoCaja	Ver sección 10 (Strategy).
ComportamientoCaja y las 3 **...Strategy** estrategias	Normal (no hace nada más), Frágil (pierde 1 de resistencia por empuje y a los 3 se rompe) y Llave (sobre un cerrojo abre los muros).	las usa Caja	Ver sección 10.
MediadorMovimiento y **MotorMovimiento** movimiento	Las reglas de movimiento: paredes, cajas, segunda caja, deslizamiento en hielo y si hubo empuje.	Motor implementa Mediador, → EstadoNivel	Ver sección 11.

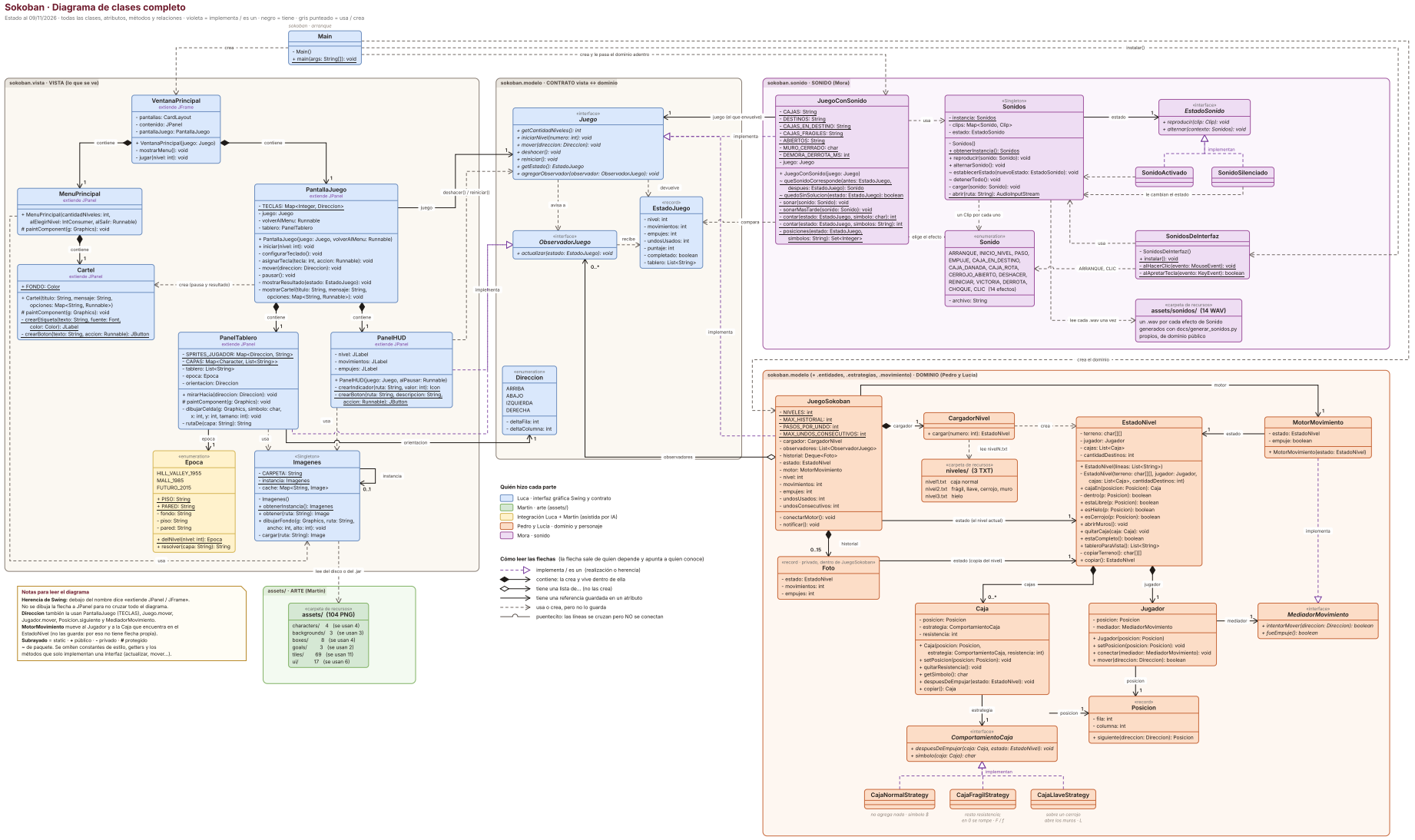
5.4 Sonido · `sokoban.sonido` (Mora)

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
JuegoConSonido	Envuelve a otro `Juego`: le pasa cada pedido y, comparando el estado de antes y de después, hace sonar el efecto que corresponde.	implementa Juego, → Juego (el envuelto), usa Sonidos y Sonido	Ver sección 13 (Decorator).
Sonidos «Singleton»	Carga los 14 .wav una sola vez y los reproduce. Esconde `javax.sound`. Si la PC no tiene audio, el juego sigue en silencio.	→ EstadoSonido, usa Sonido	Ver secciones 9 (Singleton) y 14 (State).
EstadoSonido, **SonidoActivado**, **SonidoSilenciado** de paquete	Los dos estados del sonido. Cada uno sabe qué hacer al reproducir y a cuál pasa con la tecla M.	los usa Sonidos	Ver sección 14.
Sonido enum	Los 14 efectos, cada uno con su archivo en `assets/sonidos/`.	lo usan todas las del paquete	Nada de rutas escritas a mano en otras clases.
SonidosDeInterfaz	Escucha clics y teclas en toda la aplicación: suena el arranque, el clic de cualquier botón y alterna el sonido con M.	usa Sonidos	No hizo falta tocar ninguna pantalla ni botón.

5.5 Arranque y lo que ya no se usa

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
Main	Crea el dominio, lo envuelve con el sonido, crea la ventana e instala los sonidos de la interfaz. Es la «raíz de composición»: el único lugar que conoce las clases concretas.	crea JuegoSokoban, JuegoConSonido y VentanaPrincipal; usa SonidosDeInterfaz	Usa `SwingUtilities.invokeLater`, como pide Swing.
JuegoDePrueba sin uso	El Sokoban mínimo que permitió probar la vista antes de que llegara el dominio.	nadie lo crea	Ya no se usa: conviene borrar el paquete `sokoban.prueba` antes de entregar. Por eso no aparece en los diagramas.

6 Diagrama de clases completo



docs/uml/2-diagrama-completo · es vectorial: hacé zoom en el PDF para leer los métodos. Los «puentecitos» marcan líneas que se cruzan sin conectarse.

7 Patrón Observer

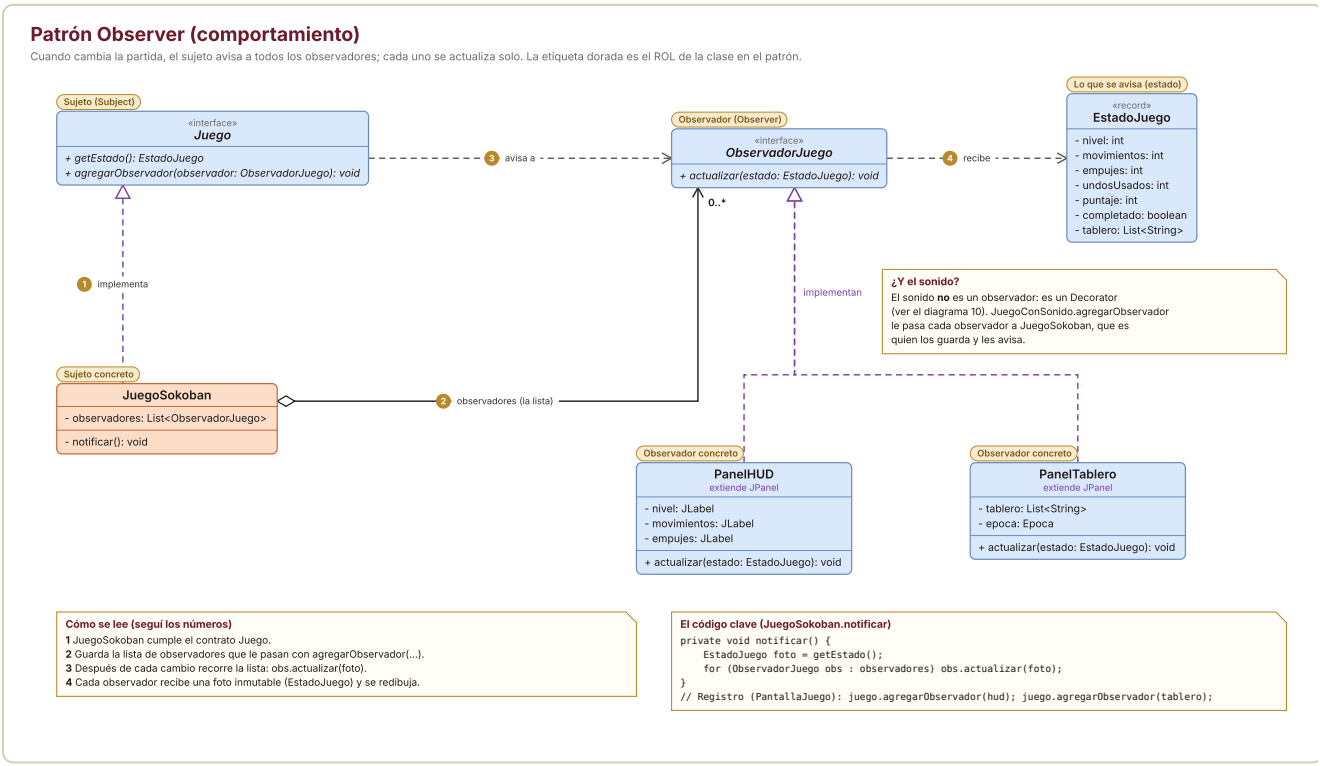
> *Sirve para que el HUD y el tablero se actualicen solos cada vez que cambia la partida, sin que el dominio sepa que existen.*

Definición

Observer es un patrón de comportamiento que define una dependencia de uno a muchos entre objetos: cuando un objeto (el sujeto) cambia de estado, todos sus dependientes (los observadores) son notificados y se actualizan automáticamente.

El problema que resuelve acá

Después de cada movimiento hay que redibujar el HUD y el tablero. Si el dominio llamara directamente a `PanelHUD` y `PanelTablero`, el dominio dependería de la vista (y de Swing). Con Observer el dominio solo conoce la interfaz `ObservadorJuego`.



docs/uml/3-patron-observer · seguí los números del 1 al 4.

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Subject (sujeto)	`Juego` (`agregarObservador`)	`modelo/Juego.java:27`
ConcreteSubject	`JuegoSokoban`	`modelo/JuegoSokoban.java:86` y `:91`
Observer	`ObservadorJuego`	`modelo/ObservadorJuego.java:7`
ConcreteObserver	`PanelHUD`, `PanelTablero`	`vista/PanelHUD.java:26`, `vista/PanelTablero.java:25`

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Estado que viaja	`EstadoJuego` (modelo «push»)	`modelo/EstadoJuego.java:12`

modelo/JuegoSokoban.java

```
85  @Override
86  public void agregarObservador(ObservadorJuego observador) {
87      observadores.add(observador);                  // guarda la INTERFAZ, no PanelHUD
88      if (estado != null) observador.actualizar(getEstado());   // lo pone al día
89  }
90
91  private void notificar() {
92      EstadoJuego foto = getEstado();                // una sola foto para todos
93      for (ObservadorJuego obs : observadores) obs.actualizar(foto);
94  }
```

7.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿Hay una interfaz de observador con un método de aviso?	`ObservadorJuego.actualizar`	✓
2	¿El sujeto permite registrar observadores?	`Juego.agregarObservador`	✓
3	¿El sujeto guarda una lista del tipo interfaz (no de clases concretas)?	`List<ObservadorJuego>` en `JuegoSokoban.java:16`	✓
4	¿Avisa después de **cada** cambio?	`notificar()` al final de `iniciarNivel`, `mover` y `deshacer` (`reiniciar` llama a `iniciarNivel`)	✓
5	¿El sujeto ignora quiénes son los observadores?	JuegoSokoban no importa nada de `sokoban.vista`	✓
6	¿Se puede sumar un observador sin modificar el sujeto?	Alcanza con implementar `ObservadorJuego` y registrarlo	✓
7	¿Se puede dar de baja un observador?	No existe `quitarObservador`	⚠
8	¿Está garantizado que cualquier implementación de Juego va a notificar?	Solo lo dice un comentario en `Juego.java`	⚠

7.2 Veredicto

Bien aplicado: están los cuatro roles, el sujeto depende solo de la interfaz y sumar un observador no obliga a tocar nada. Para defenderlo, conviene saber estos matices:

- El sonido **no** es un observador: es un Decorator (sección 13). `JuegoConSonido.agregarObservador` le pasa cada observador a `JuegoSokoban`, que es quien los guarda y les avisa.

- En el GoF el sujeto también tiene `detach` (dar de baja). Acá no hace falta porque el HUD y el tablero viven toda la ejecución.
- Cada implementación de `Juego` tiene que acordarse de llamar a `notificar()`. Una mejora posible es una clase abstracta (por ejemplo `JuegoObservable`) que ya traiga la lista y `notificar()`.
- Si el movimiento choca contra una pared, `JuegoSokoban` no avisa (no cambió nada). Es correcto: un observador solo se entera de cambios.
- `PantallaJuego` no es observador: después de mover consulta `getEstado()` para ver si ganó. Es válido (consulta directa), porque hoy un nivel solo se completa dentro de un `mover` (también cuando la caja desliza en el hielo).

8 Patrón Facade

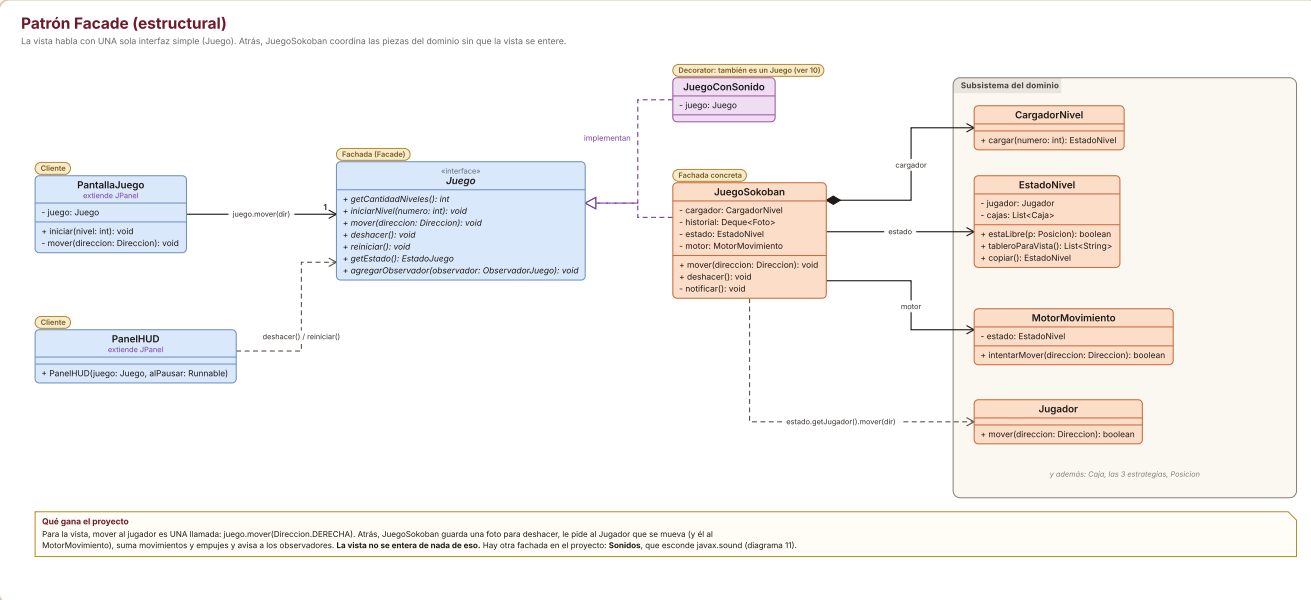
> *Sirve para que la vista hable con el dominio a través de una sola puerta simple, sin conocer las piezas que hay detrás.*

Definición

Facade es un patrón estructural que proporciona una interfaz unificada y simple a un conjunto de interfaces de un subsistema, para que el subsistema sea más fácil de usar.

El problema que resuelve acá

Mover al jugador implica sacar una foto para deshacer, mirar el tablero, empujar la caja, ver si es frágil o llave, dejarla deslizar en el hielo, contar movimientos y empujes y avisar a los observadores. La vista solo dice `juego.mover(Direccion.DERECHA)`.



docs/uml/4-patron-facade · ahora el subsistema existe (naranja).

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Client	`PantallaJuego`, `PanelHUD`, `VentanaPrincipal`	usan solo métodos de `Juego`

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Facade	`Juego` (interfaz) + `JuegoSokoban` (la implementa)	`modelo/Juego.java:13`, `modelo/JuegoSokoban.java:9`
Subsystem classes	`CargadorNivel`, `EstadoNivel`, `MotorMovimiento`, `Jugador`, `Caja` y las estrategias	`modelo/` y sus subpaquetes

8.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿Hay un punto de entrada único y con pocas operaciones?	`Juego`: 7 métodos	✓
2	¿Los clientes usan solo la fachada?	La vista no importa nada del dominio	✓
3	¿Hay detrás un subsistema de varias clases?	11 clases e interfaces del dominio	✓
4	¿La fachada delega en el subsistema en vez de resolver todo?	`JuegoSokoban.mover` le pide el movimiento al Jugador (`:45`), la carga al cargador (`:29`)	✓
5	¿La interfaz tiene una sola responsabilidad?	`agregarObservador` mezcla «fachada» y «sujeto»	⚠

8.2 Veredicto

- Ahora el patrón está completo: hay una puerta chica y clara, y atrás un subsistema real que la vista nunca ve (GRASP «Variaciones protegidas»).
- `JuegoSokoban` coordina y además se queda con tres tareas propias: el historial de deshacer, los contadores y el puntaje. Son pocas líneas y tiene sentido que estén ahí (es quien sabe cuándo pasó un movimiento).
- En el GoF la fachada suele ser una clase concreta. Acá es una interfaz más una clase que la implementa: se combina Facade con DIP, y eso es lo que permitió meter el sonido «adelante» sin que la vista cambie.
- Hay una segunda fachada en el proyecto: `Sonidos` esconde `javax.sound` (`AudioSystem`, `Clip`, formatos, excepciones) detrás de `reproducir(Sonido.PASO)`.
- Si te preguntan por ISP: `Juego` podría dividirse en `Juego` (acciones) y `JuegoObservable` (registrar observadores). Con 7 métodos no es grave, pero es bueno saberlo.

9 Patrón Singleton (dos veces)

> *Sirve para que haya una única caché de imágenes y un único reproductor de sonidos, y cada archivo se lea una sola vez.*

Definición

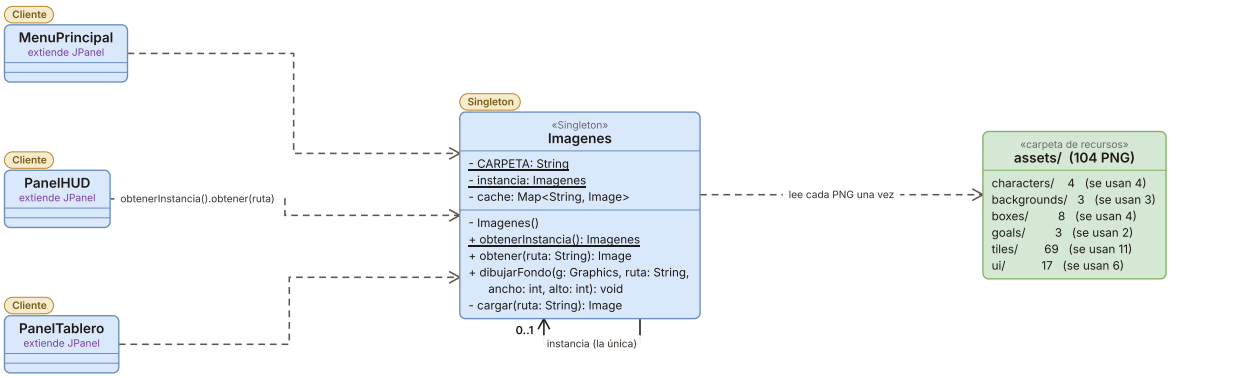
Singleton es un patrón creacional que garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

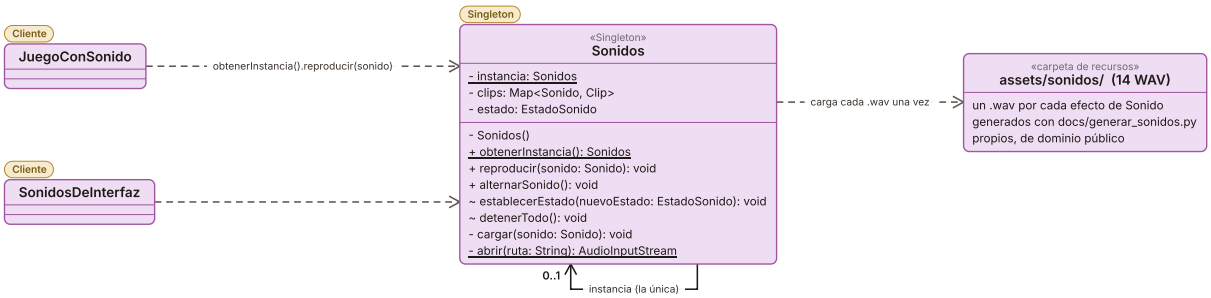
El problema que resuelve acá

El tablero se redibuja en cada movimiento y usa decenas de imágenes; el sonido usa 14 efectos que pueden sonar muchas veces por segundo. Leer los archivos cada vez sería lento, y tener una copia por pantalla duplicaría memoria. Con una sola instancia, cada archivo se carga una vez.

Patrón Singleton (creacional)

Hay DOS: una sola caché de imágenes (Imagenes) y un solo reproductor de sonidos (Sonidos). Todos los piden con obtenerInstancia().





Las 3 piezas que tiene que tener un Singleton (las dos clases las tienen)

```
private static Imagenes instancia;          // 1. la única instancia, guardada en la clase   (Sonidos: igual)
private Imagenes() { }                      // 2. constructor privado: nadie más puede hacer new
public static Imagenes obtenerInstancia() {  // 3. punto de acceso global
    if (instancia == null) instancia = new Imagenes();   // se crea la primera vez que se pide
    return instancia; }
```

docs/uml/5-patron-singleton · arriba Imagenes (Luca), abajo Sonidos (Mora).

sonido/Sonidos.java

```
31  public final class Sonidos {                    // final: nadie puede heredarla
32
33      private static Sonidos instancia;           // 1. la única instancia
34
35      private final Map<Sonido, Clip> clips = new EnumMap<>(Sonido.class);
36      private EstadoSonido estado = new SonidoActivado();
```

En la línea 38 está el constructor privado (2), que carga los 14 efectos. Y el acceso global (3):

sonido/Sonidos.java

```
49  public static Sonidos obtenerInstancia() {      // 3. acceso global
50      if (instancia == null) {                    //    se crea la primera vez (lazy)
51          instancia = new Sonidos();
52      }
53      return instancia;
54  }
```

9.1 Lista de chequeo

#	Pregunta	Imagenes	Sonidos	Estado
1	¿El constructor es privado?	`vista/Imagenes.java:26`	`sonido/Sonidos.java:38`	✓
2	¿Hay un atributo `static` con la única instancia?	`:22`	`:33`	✓
3	¿Hay un método `static` de acceso que la crea si no existe?	`:29`	`:49`	✓
4	¿La clase es `final`?	`:18`	`:31`	✓
5	¿Realmente hace falta una sola instancia?	caché compartida	los Clip se cargan una vez y el estado (silencio) es uno solo	✓
6	¿Es seguro con varios hilos?	No está sincronizado, pero las dos se usan solo desde el hilo de Swing		⚠ ver abajo

9.2 Veredicto

- Bien aplicado y bien justificado en los dos casos: tienen las tres piezas y un motivo real. `Sonidos` además combina Singleton con Facade (esconde `javax.sound`) y es el contexto del State (sección 14).
- Ninguno es seguro con varios hilos, pero en Swing todo corre en un único hilo (el EDT): las teclas, los botones y el `javax.swing.Timer` que usa el sonido de derrota corren ahí. Hoy no hay riesgo.
- `Sonidos` no rompe el juego si la PC no tiene audio: atrapa el error, avisa por consola y sigue en silencio (`sonido/Sonidos.java:43`).
- Críticas que te pueden hacer: es un estado global y dificulta los tests. La alternativa sería crear una sola instancia en `Main` y pasarla por constructor. Para una caché y un reproductor, el Singleton es defendible.

- En GRASP, `Imagenes` y `Sonidos` son «Fabricación pura»: no representan nada del juego, existen para resolver un tema técnico.

10 Patrón Strategy: las tres cajas

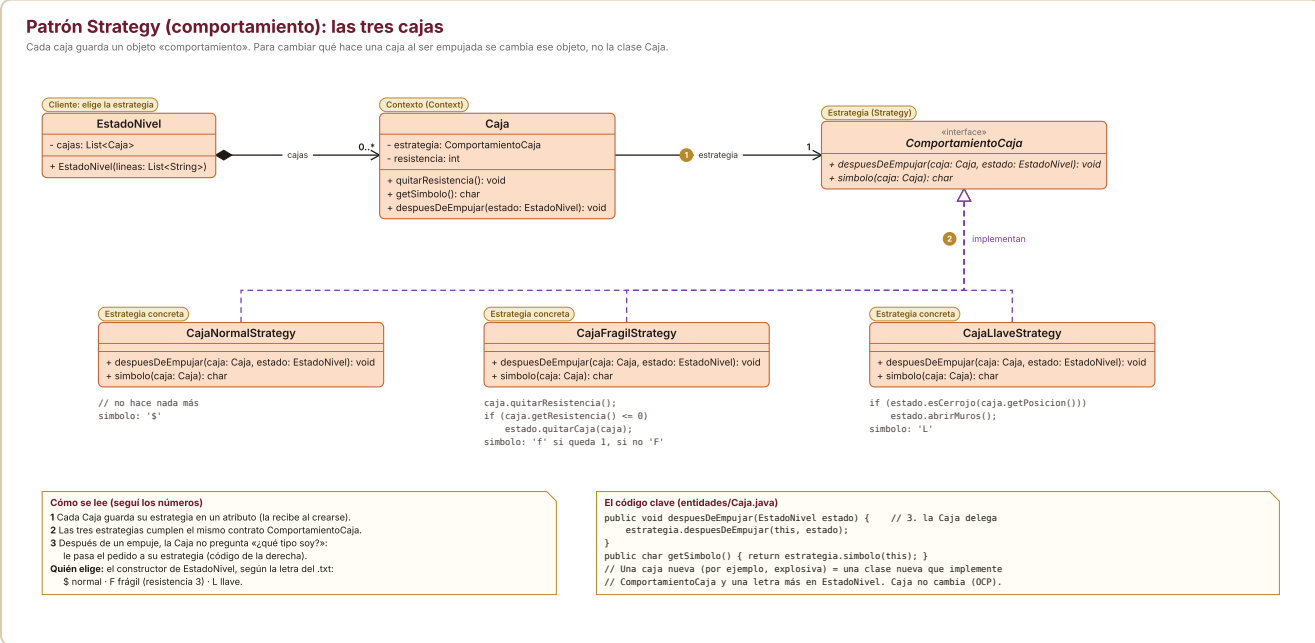
> *Sirve para que cada tipo de caja reaccione distinto a un empuje sin llenar de «if» la clase Caja ni el motor de movimiento.*

Definición

Strategy es un patrón de comportamiento que define una familia de algoritmos, encapsula cada uno y los hace intercambiables. Permite que el algoritmo varíe sin cambiar a los clientes que lo usan.

El problema que resuelve acá

«Cajas normales, cajas frágiles y cajas llave» se mueven igual, pero después de un empuje hacen cosas distintas: la frágil pierde resistencia y se rompe, la llave abre los muros si queda sobre un cerrojo. Sin el patrón, `Caja` o `MotorMovimiento` tendrían un `if (tipo == FRAGIL) … else if (tipo == LLAVE) …` que crece con cada caja nueva.



docs/uml/8-patron-strategy · debajo de cada estrategia, lo que hace su código.

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Context	`Caja` (guarda la estrategia y le delega)	`modelo/entidades/Caja.java:9` y `:24`
Strategy	`ComportamientoCaja`	`modelo/estrategias/ComportamientoCaja.java:6`
ConcreteStrategy	`CajaNormalStrategy`, `CajaFragilStrategy`, `CajaLlaveStrategy`	`modelo/estrategias/`

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Quién elige la estrategia	el constructor de `EstadoNivel`, según la letra del .txt (`$`, `F`, `L`)	`modelo/EstadoNivel.java:40` a `:51`

`modelo/estrategias/CajaFragilStrategy.java`

```
7   @Override
8   public void despuesDeEmpujar(Caja caja, EstadoNivel estado) {
9       caja.quitarResistencia();                    // arranca en 3 (EstadoNivel)
10      if (caja.getResistencia() <= 0) {
11          estado.quitarCaja(caja);                 // se rompe y desaparece del tablero
12      }
13  }
```

10.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿Hay una interfaz común para los comportamientos?	`ComportamientoCaja`: `despuesDeEmpujar` y `simbolo`	✓
2	¿El contexto guarda la estrategia en un atributo del tipo interfaz?	`Caja.java:9`: `ComportamientoCaja estrategia`	✓
3	¿El contexto delega en vez de preguntar «qué tipo soy»?	`Caja.java:22` y `:24`; no hay `instanceof` ni `switch` por tipo	✓
4	¿Las estrategias no guardan datos propios?	La resistencia vive en la Caja, por eso `Caja.copiar()` puede compartir la estrategia (`:29`)	✓
5	¿Una caja nueva no obliga a tocar Caja ni MotorMovimiento?	Una clase nueva + una letra más en `EstadoNivel`	✓
6	¿Se puede cambiar la estrategia mientras se juega?	Es `final` y no hay setter	⚠ no hace falta

10.2 Veredicto

- Bien aplicado: el ejemplo de libro del patrón. Cada comportamiento está en su clase y `Caja` no sabe qué tipo de caja es.
- El único lugar que conoce las tres clases es el constructor de `EstadoNivel`, que elige según la letra del archivo. Si crecieran los tipos, ese `switch` podría pasar a una pequeña fábrica (Factory Method).
- El símbolo también es parte de la estrategia: la frágil se dibuja `F` o `f` según la resistencia, y `EstadoNivel` la cambia a `G` o `g` si está sobre un destino.
- La llave abre **todos** los muros `X` del nivel (`EstadoNivel.abrirMuros`); la consigna habla de «los muros correspondientes». Con un solo cerrojo por nivel da lo mismo; si un nivel tuviera dos cerrojos, habría que decidir qué muro abre cada uno.

11 Patrón Mediator: el movimiento

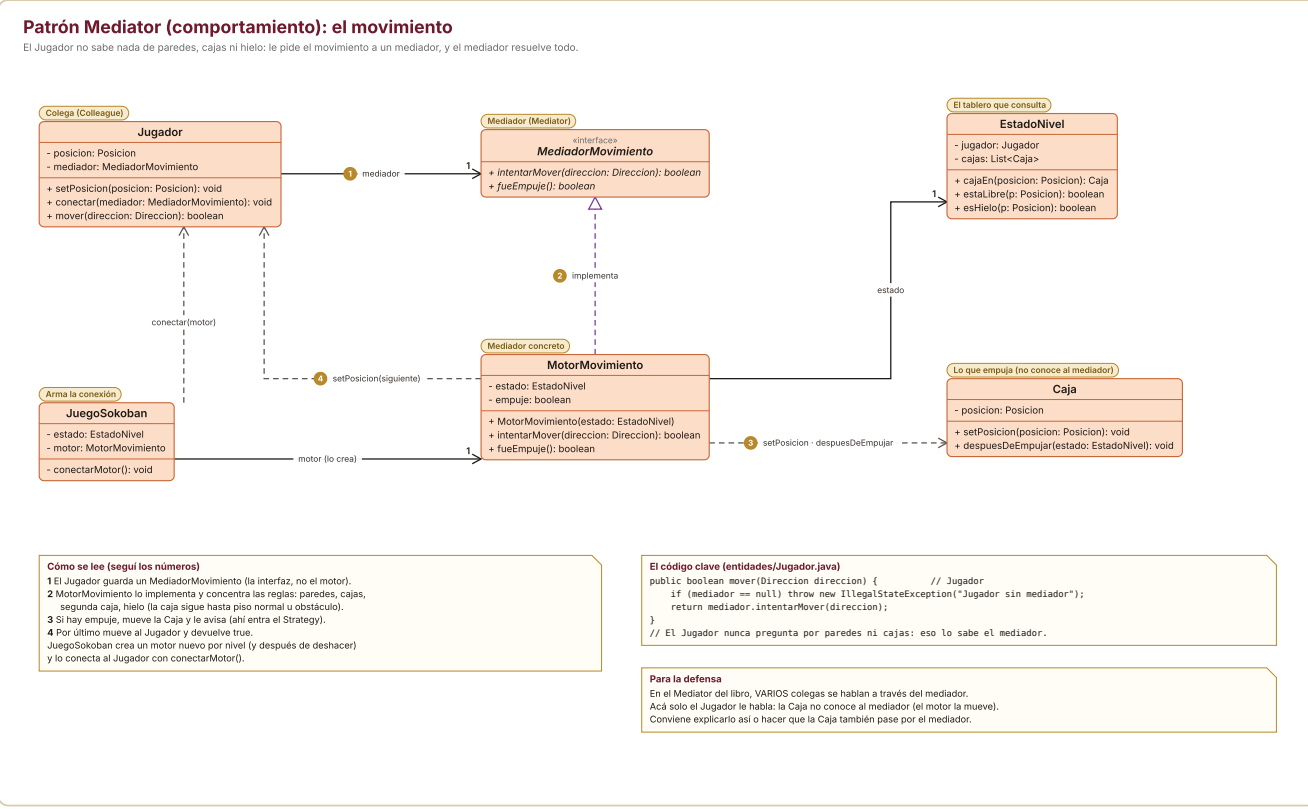
> *Sirve para que el jugador no tenga que saber nada de paredes, cajas ni hielo: le pide el movimiento a un mediador que conoce las reglas.*

Definición

Mediator es un patrón de comportamiento que define un objeto que encapsula cómo interactúa un conjunto de objetos. Promueve el bajo acoplamiento, porque evita que los objetos se refieran unos a otros directamente.

El problema que resuelve acá

Un paso del jugador involucra al jugador, a la caja de adelante, a lo que hay detrás de la caja y al terreno (hielo). Si el `Jugador` empujara directamente a la `Caja` y cada uno revisara el tablero, las reglas quedarían repartidas. Con el mediador, todas las reglas de movimiento están en `MotorMovimiento`.



docs/uml/9-patron-mediator · seguí los números del 1 al 4.

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Mediator	`MediadorMovimiento`	`modelo/movimiento/MediadorMovimiento.java:6`
ConcreteMediator	`MotorMovimiento`	`modelo/movimiento/MotorMovimiento.java:10`
Colleague	`Jugador` (le habla al mediador)	`modelo/entidades/Jugador.java:9` y `:22`
Otro participante	`Caja` (la mueve el mediador, pero ella no lo conoce)	`MotorMovimiento.java:38` y `:39`

modelo/movimiento/MotorMovimiento.java

```
29  Posicion destinoCaja = siguiente.siguiente(direccion);
30  if (!estado.estaLibre(destinoCaja)) return false; // Ni pared ni segunda caja.
31
32  // Si entra al hielo, sigue hasta el primer piso normal o un obstaculo.
33  while (estado.esHielo(destinoCaja)
34          && estado.estaLibre(destinoCaja.siguiente(direccion))) {
35      destinoCaja = destinoCaja.siguiente(direccion);
36  }
37
38  caja.setPosicion(destinoCaja);
39  caja.despuesDeEmpujar(estado); // STRATEGY: fragil o llave aplican su efecto.
40  jugador.setPosicion(siguiente);
```

11.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿Hay una interfaz de mediador?	`MediadorMovimiento`: `intentarMover`, `fueEmpuje`	✓
2	¿El colega conoce solo la interfaz, no el mediador concreto?	`Jugador.java:9`	✓
3	¿Las reglas están concentradas en el mediador?	`MotorMovimiento.java:17` a `:43`	✓
4	¿Los participantes no se conocen entre sí?	`Jugador` no menciona a `Caja` ni al revés	✓
5	¿Todos los colegas hablan a través del mediador?	Solo el Jugador; la Caja no lo conoce	⚠
6	¿Está claro quién crea y conecta el mediador?	`JuegoSokoban.conectarMotor()` (`:36`), al iniciar y al deshacer	✓

11.2 Veredicto

- Bien encaminado: las reglas de movimiento están en un solo lugar y el Jugador quedó muy simple.
- En el Mediator del libro, varios colegas se hablan entre ellos a través del mediador. Acá solo el Jugador le habla; la Caja es movida por el motor sin conocerlo. Para la defensa hay dos caminos: explicarlo como «el motor coordina al jugador, las cajas y el tablero, que nunca se hablan entre sí», o hacer que la Caja también pase por el mediador.
- ¿Por qué se vuelve a conectar el motor al deshacer? Porque la foto restaurada trae un `Jugador` nuevo (lo crea `EstadoNivel.copiar()`) que todavía no tiene mediador. Si no se conectara, el Jugador avisa con `IllegalStateException("Jugador sin mediador")` .
- Detalle de DIP: `JuegoSokoban` guarda el motor como `MotorMovimiento` (la clase), aunque solo usa `fueEmpuje()` , que ya está en la interfaz. Podría declararse como `MediadorMovimiento` .

12 Deshacer con fotos del nivel (idea de Memento)

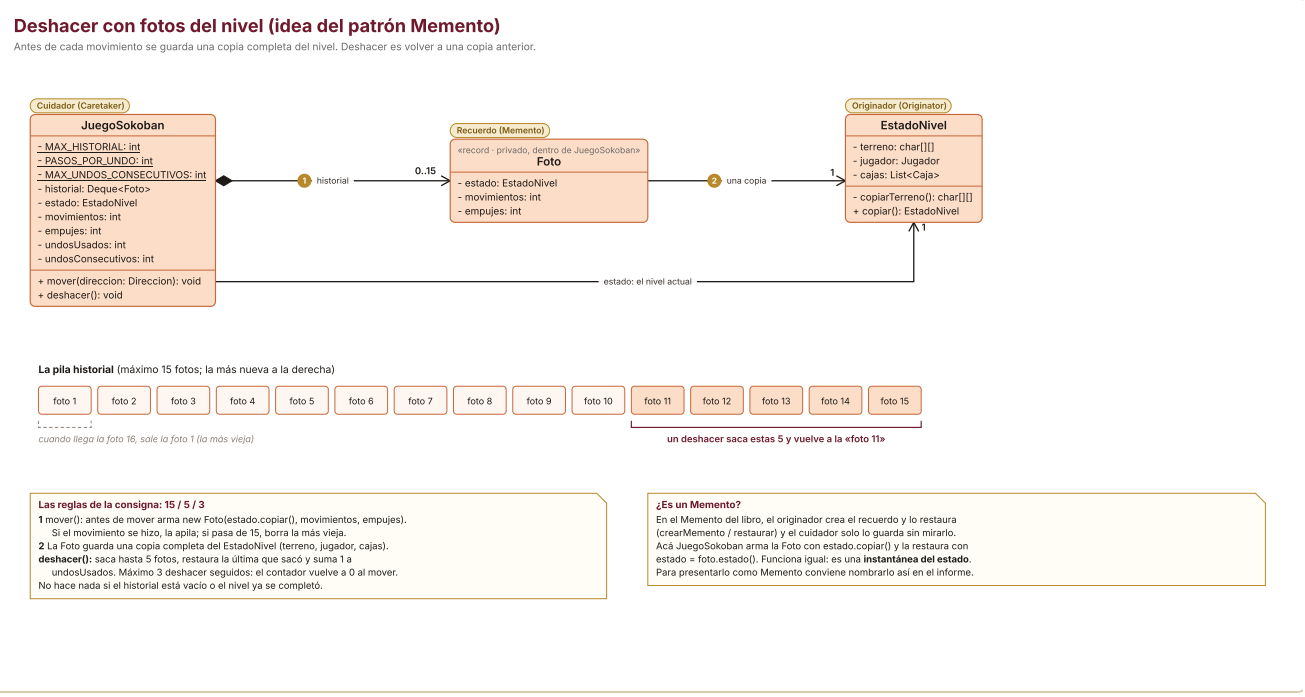
> *Sirve para cumplir la regla 15 / 5 / 3 de la consigna: antes de cada movimiento se guarda una copia del nivel, y deshacer es volver a una copia anterior.*

Definición

Memento es un patrón de comportamiento que, sin romper el encapsulamiento, captura el estado interno de un objeto y lo guarda afuera para poder restaurarlo más tarde.

La regla de la consigna

«Se permite deshacer los últimos 15 movimientos realizados por el jugador. Para ello, debe existir un botón undo que, cada vez que se presione, restaure el estado del nivel 5 movimientos hacia atrás, hasta un máximo de tres usos consecutivos.»



docs/uml/12-deshacer-memento · abajo, cómo se ve la pila de fotos.

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Originator	`EstadoNivel` (sabe copiarse)	`modelo/EstadoNivel.java:152`
Memento	`Foto` (copia del nivel + movimientos + empujes)	`modelo/JuegoSokoban.java:96`
Caretaker	`JuegoSokoban` (la pila `historial`)	`modelo/JuegoSokoban.java:17`

modelo/JuegoSokoban.java

```
56  public void deshacer() {
57      if (estado == null || estado.estaCompleto() || historial.isEmpty()
58              || undosConsecutivos >= MAX_UNDOS_CONSECUTIVOS) return;   // máximo 3 seguidos
59      Foto foto = null;
60      for (int i = 0; i < PASOS_POR_UNDO && !historial.isEmpty(); i++) {
61          foto = historial.pop();                    // saca hasta 5 fotos
62      }
63      estado = foto.estado();                        // vuelve a la última que sacó
64      movimientos = foto.movimientos();
65      empujes = foto.empujes();
66      undosUsados++;
67      undosConsecutivos++;
68      conectarMotor();
69      notificar();
70  }
```

12.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿Se guarda una foto antes de cada movimiento, y solo si el movimiento se hizo?	`JuegoSokoban.java:44` a `:47`	✓
2	¿Se guardan como máximo 15?	`:48` borra la más vieja	✓
3	¿Cada deshacer vuelve 5 movimientos?	`:60` a `:63` (si hay menos, vuelve hasta donde puede)	✓
4	¿Máximo 3 seguidos, y se reinicia al mover?	`:57`, `:58` y `:51`	✓
5	¿La foto es una copia de verdad y no el mismo objeto?	`EstadoNivel.copiar()` copia terreno, jugador y cada caja	✓
6	¿Se cuenta cuántas veces se usó, para el resumen?	`undosUsados` (`:66`) va a `EstadoJuego` y al puntaje	✓
7	¿Está presentado como Memento?	El código no lo nombra así	⚠

12.2 Veredicto

- Cumple las tres reglas de la consigna y es fácil de seguir: guardar una foto entera es más simple que deshacer cada movimiento al revés.
- En el Memento del libro, el originador crea el recuerdo y lo restaura (`crearMemento` / `restaurar`), y el cuidador lo guarda sin mirarlo. Acá `JuegoSokoban` arma la `Foto` con `estado.copiar()` y la restaura reemplazando `estado`. Es la misma idea (una instantánea del estado): para presentarlo como patrón, conviene nombrarlo así en el informe.
- Se guardan hasta 15 copias completas del nivel. Con niveles chicos no pesa nada.
- Una vez completado el nivel ya no se puede deshacer, y deshacer cuesta 50 puntos (sección 4.4).

13 Patrón Decorator: el sonido

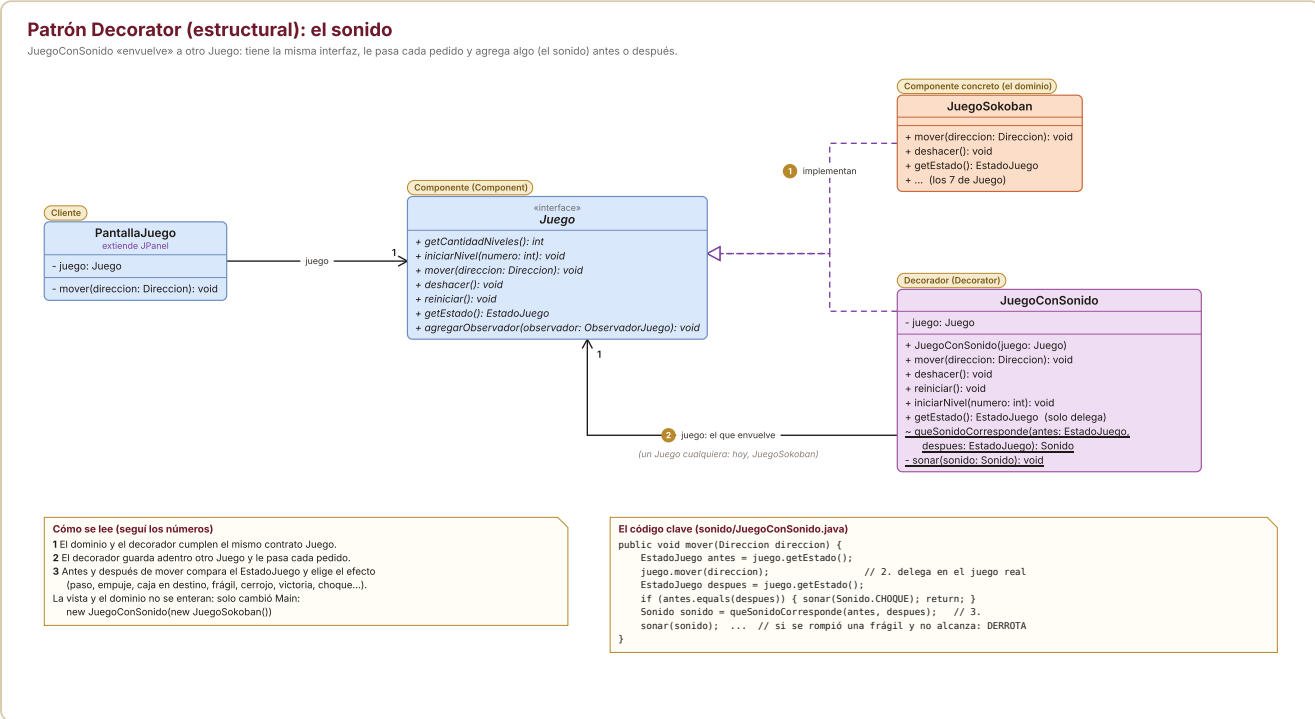
> *Sirve para agregarle sonido al juego sin tocar el dominio ni la vista: el sonido «envuelve» al juego real.*

Definición

Decorator es un patrón estructural que agrega responsabilidades a un objeto envolviéndolo en otro objeto con la misma interfaz. Es una alternativa flexible a crear subclases.

El problema que resuelve acá

«las acciones del juego deben estar acompañadas de efectos de sonido apropiados» Poner llamadas de sonido adentro de `JuegoSokoban` mezclaría reglas con audio. `JuegoConSonido` se pone «adelante» del dominio, deja pasar cada pedido y agrega el sonido.



docs/uml/10-patron-decorator · el decorador implementa Juego y además guarda un Juego adentro.

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Component	`Juego`	`modelo/Juego.java:13`
ConcreteComponent	`JuegoSokoban`	`modelo/JuegoSokoban.java:9`
Decorator	`JuegoConSonido` (no hay decorador abstracto: con uno solo no hace falta)	`sonido/JuegoConSonido.java:27` y `:38`
Client	la vista, que recibe un `Juego`	`Main.java:19`

sonido/JuegoConSonido.java

```
52  @Override
53  public void mover(Direccion direccion) {
54      EstadoJuego antes = juego.getEstado();
55      juego.mover(direccion);                        // delega en el juego real
56      EstadoJuego despues = juego.getEstado();
57      if (antes.equals(despues)) {
58          sonar(Sonido.CHOQUE);          // pared, caja trabada: no se pudo mover
59          return;
60      }
61      Sonido sonido = queSonidoCorresponde(antes, despues);
62      sonar(sonido);
63      if (sonido == Sonido.CAJA_ROTA && quedoSinSolucion(despues)) {
64          sonarMasTarde(Sonido.DERROTA);
65      }
66  }
```

13.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿El decorador implementa la misma interfaz que el componente?	`JuegoConSonido.java:27`	✓
2	¿Guarda el componente envuelto con el tipo interfaz?	`:38` : `private final Juego juego`	✓
3	¿Lo recibe desde afuera (constructor)?	`:40`	✓
4	¿Delega **todos** los métodos?	los que suenan (`:47` a `:79`) y los que solo delegan (`:84` a `:96`)	✓
5	¿El componente no sabe que está decorado?	`JuegoSokoban` no importa nada de `sokoban.sonido`	✓
6	¿Se puede poner o sacar sin tocar a los clientes?	Solo cambia `Main.java:19`	✓
7	¿Depende de detalles del componente?	Lee los símbolos del tablero (`:30` a `:35`)	⚠

13.2 Veredicto

- Bien aplicado y muy fácil de mostrar en la defensa: una línea en `Main` pone o saca todo el sonido.
- Para saber qué pasó, compara el tablero de antes y de después: si bajaron las cajas frágiles, se rompió una; si aparecieron muros abiertos, se abrió el cerrojo; si subieron las cajas en destino, sonó el «ding». Por eso depende de los símbolos que arma el dominio: si el dominio cambia una letra, hay que actualizar las constantes de `JuegoConSonido`.
- «Perder» se detecta acá: si se rompió una frágil y ya no alcanzan las cajas para los destinos, suena «derrota» 600 ms después. No hay cartel que lo diga (ver «Para revisar en equipo»).
- En el GoF suele haber un decorador abstracto del que heredan varios decoradores. Con uno solo no hace falta; si mañana hubiera otro (por ejemplo, uno que guarde estadísticas), se podrían apilar: `new JuegoConSonido(new OtroDecorador(new JuegoSokoban()))`.

14 Patrón State: sonido activado o silenciado

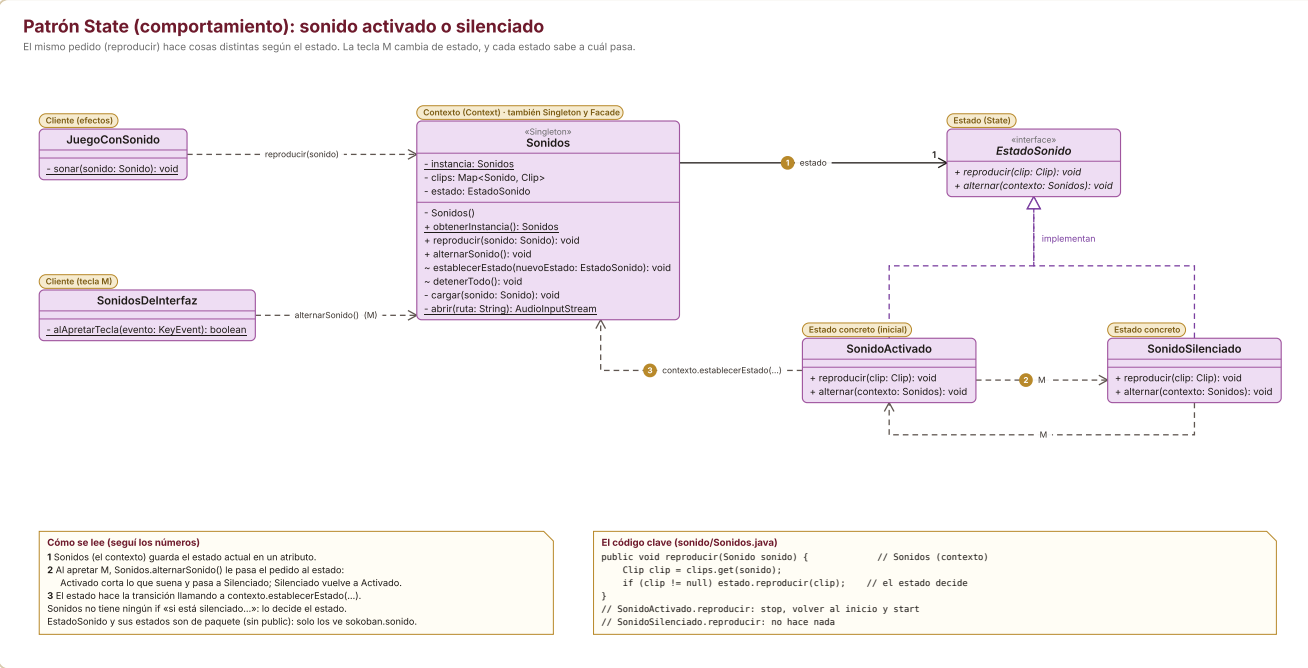
> *Sirve para que el mismo pedido («reproducí este sonido») haga cosas distintas según si el sonido está activado o silenciado, sin un «if» en el reproductor.*

Definición

State es un patrón de comportamiento que permite que un objeto cambie su comportamiento cuando cambia su estado interno; parece que el objeto cambiara de clase.

El problema que resuelve acá

Con la tecla M el jugador silencia o activa el sonido. Sin el patrón, `Sonidos` tendría un `boolean` `silenciado` y un `if` en cada método. Con State, cada estado sabe qué hacer y a cuál pasa.



docs/uml/11-patron-state · las flechas «M» son las transiciones.

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Context	`Sonidos` (guarda el estado actual)	`sonido/Sonidos.java:36`
State	`EstadoSonido` (de paquete)	`sonido/EstadoSonido.java:17`
ConcreteState	`SonidoActivado` (el inicial), `SonidoSilenciado`	`sonido/SonidoActivado.java:10`, `sonido/SonidoSilenciado.java:10`
Quién dispara el cambio	`SonidosDeInterfaz` (tecla M)	`sonido/SonidosDeInterfaz.java:46`

sonido/SonidoActivado.java

```
13  @Override
14  public void reproducir(Clip clip) {
15      clip.stop();
16      clip.setFramePosition(0);
17      clip.start();
18  }
19
20  /** Al silenciar, corta lo que esté sonando y pasa a Silenciado. */
21  @Override
22  public void alternar(Sonidos contexto) {
23      contexto.detenerTodo();
24      contexto.establecerEstado(new SonidoSilenciado());   // la decide el estado
25  }
```

14.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿Hay una interfaz de estado con las operaciones que cambian?	`EstadoSonido` : `reproducir` , `alternar`	✓
2	¿El contexto guarda el estado actual en un atributo del tipo interfaz?	`Sonidos.java:36`	✓
3	¿El contexto delega en el estado en vez de usar «if»?	`reproducir` (`:57`) y `alternarSonido` (`:65`)	✓
4	¿Las transiciones están adentro de los estados?	`SonidoActivado.java:22` , `SonidoSilenciado.java:20`	✓
5	¿Solo los estados pueden cambiar el estado?	`establecerEstado` es de paquete (`:70`)	✓

14.2 Veredicto

- Bien aplicado y chico: es un buen ejemplo para explicar State en dos minutos.
- **«Cada equipo debe, necesariamente, incorporar dos funcionalidades adicionales al juego que impliquen la aplicación de patrones de diseño.»** Silenciar con M usando State puede contar como una de las dos. Falta definir la otra.
- Cada cambio crea un objeto de estado nuevo (`new SonidoSilenciado()`). Como los estados no guardan datos, se podrían reutilizar dos instancias fijas ; con una tecla que se aprieta de vez en cuando no importa.

15 MVC y el DTO `EstadoJuego`

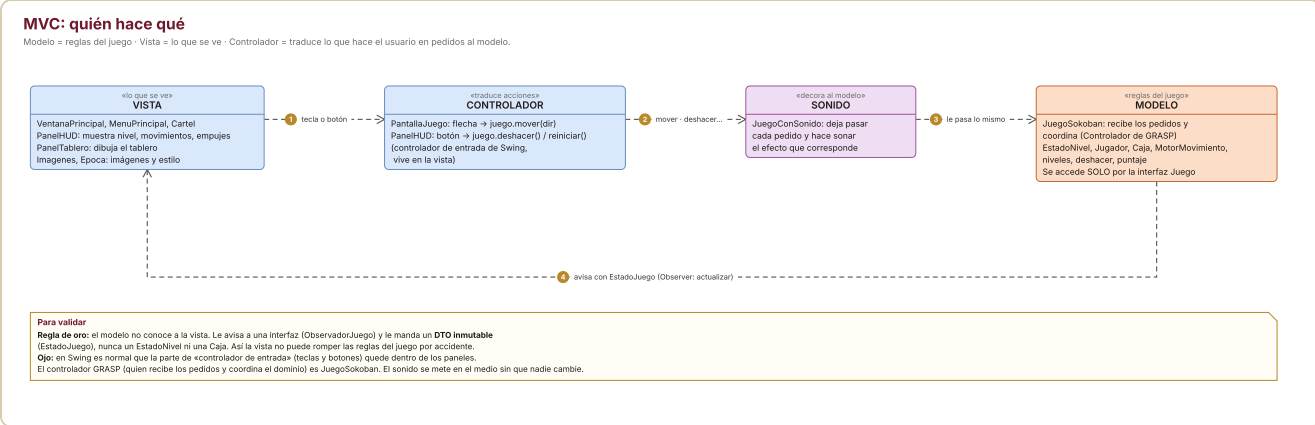
> *MVC no es un patrón GoF sino una forma de ordenar la aplicación: separa las reglas del juego (modelo) de lo que se ve (vista) y de cómo entran las acciones (controlador).*

Definición

MVC (Modelo-Vista-Controlador) divide la aplicación en el modelo, que tiene los datos y las reglas; la vista, que los muestra; y el controlador, que recibe las acciones del usuario y las traduce en operaciones sobre el modelo.

DTO

Un DTO (objeto de transferencia de datos) es un objeto simple, sin lógica, que solo lleva datos de una capa a otra. Acá es `EstadoJuego`: la vista recibe una foto del nivel y nunca un `EstadoNivel` o una `Caja` del dominio.



docs/uml/6-mvc · el sonido se mete en el medio sin que nadie cambie.

15.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿El modelo no conoce a la vista?	Sin imports de `sokoban.vista` en modelo ni en sonido	✓
2	¿La vista recibe datos inmutables?	`record` + `List.copyOf`	✓
3	¿La vista modifica el modelo solo a través de operaciones?	Solo llama métodos de `Juego`	✓
4	¿Está claro quién es el controlador?	Dos niveles: ver abajo	✓
5	¿El formato del tablero alcanza para todo lo que hay que dibujar?	Un carácter por celda	⚠

- En Swing es normal que el «controlador de entrada» (teclas y botones) viva dentro de los paneles: `PantallaJuego` traduce las flechas y `PanelHUD` los botones.
- El «Controlador» de GRASP (la clase que recibe los pedidos del sistema y coordina el dominio) es `JuegoSokoban`. Si te preguntan «¿dónde está el controlador?», explicá los dos niveles.

- El equipo eligió sumar símbolos para las celdas con dos cosas encima (`f` , `G` , `g` , `K` , `c`). Todavía quedan casos sin símbolo: la caja llave sobre el cerrojo tapa el dibujo del cerrojo, y el jugador o una caja sobre hielo tapan el hielo.

16 SOLID y GRASP, con evidencia

> *Cada principio con el lugar exacto del código que lo muestra, y también dónde está flojo: en la defensa suma más reconocer un límite que exagerar.*

16.1 SOLID

Principio	Dónde se ve	Dónde está flojo	Estado
SRP · una sola responsabilidad	Leer el archivo (`CargadorNivel`), guardar el tablero (`EstadoNivel`), mover (`MotorMovimiento`), reaccionar al empuje (estrategias), sonar (`Sonidos`): cada cosa en su clase.	`PantallaJuego` arma la partida, maneja el teclado y muestra los carteles. `JuegoSokoban` junta historial, contadores y puntaje (son pocas líneas).	⚠
OCP · abierto a extensión, cerrado a modificación	Caja nueva = estrategia nueva; sonido = decorador nuevo; nuevo observador, símbolo (`CAPAS`) o época (`Epoca`) sin tocar la lógica existente.	Una caja nueva igual suma una letra en el `switch` de `EstadoNivel` .	✓
LSP · sustitución de Liskov	`JuegoSokoban` y `JuegoConSonido` son intercambiables detrás de `Juego` : la vista funciona igual con cualquiera.	Las tres estrategias también son intercambiables para `Caja` .	✓
ISP · interfaces chicas	`ObservadorJuego` tiene un método; `MediadorMovimiento` , `ComportamientoCaja` y `EstadoSonido` , dos.	`Juego` mezcla acciones con el registro de observadores.	✓
DIP · depender de abstracciones	La vista depende de `Juego` ; el Jugador de `MediadorMovimiento` ; la Caja de `ComportamientoCaja` ; el sonido de `Juego` . Solo `Main` conoce las clases concretas.	`JuegoSokoban` guarda el motor como `MotorMovimiento` y no como la interfaz.	✓

16.2 GRASP

Patrón GRASP	Dónde se ve en el proyecto	Estado
Experto en información	`EstadoNivel` sabe si una celda está libre y si se ganó; `Direccion` sabe cuánto se mueve; `Epoca` sabe qué piso y pared le tocan.	✓
Creador	`EstadoNivel` crea el Jugador y las Cajas que contiene; `CargadorNivel` crea el `EstadoNivel` ; `PantallaJuego` crea el HUD y el tablero.	✓
Controlador	`JuegoSokoban` recibe los pedidos del sistema (iniciar, mover, deshacer, reiniciar) y coordina.	✓

Patrón GRASP	Dónde se ve en el proyecto	Estado
Bajo acoplamiento	Vista, dominio y sonido no se conocen; las pantallas se avisan con `Runnable` / `IntConsumer`.	✓
Alta cohesión	Clases cortas con una sola tarea: la más larga en uso es `JuegoConSonido` (169 líneas, con comentarios).	✓
Polimorfismo	Las cajas (Strategy), los estados del sonido (State) y los observadores se usan sin preguntar de qué clase son.	✓
Fabricación pura	`Imagenes`, `Sonidos`, `CargadorNivel` y `MotorMovimiento`: no son parte del «mundo» del juego, resuelven un tema técnico o concentran reglas.	✓
Indirección / Variaciones protegidas	`Juego` se interpone entre vista y dominio; `MediadorMovimiento` entre el jugador y las reglas.	✓

17 Lo que falta y para revisar en equipo

> *Lo que todavía no está, en orden de importancia, y lo que vimos al juntar las partes. Nada de esto rompe el juego: son cosas para charlar y dejar prolijas antes de entregar.*

17.1 Qué ya se cumple de la consigna (probado en pantalla con los 3 niveles)

✓ niveles .txt · ✓ caja normal, frágil y llave · ✓ piso resbaladizo · ✓ cerrojo y muros · ✓ deshacer 15 / 5 / 3 · ✓ victoria y paso al nivel siguiente · ✓ resumen con movimientos, empujes, deshacer y puntaje · ✓ HUD · ✓ sonidos propios de dominio público · ✓ imágenes · ✓ README

17.2 Qué falta para entregar

Tema	Qué hay que hacer	Prioridad
Dos funcionalidades extra	**«Cada equipo debe, necesariamente, incorporar dos funcionalidades adicionales al juego que impliquen la aplicación de patrones de diseño.»** El silencio con la tecla M (State) puede contar como una. Falta la otra.	alta
StarUML	**«Diagrama de clases en StarUML»** Estos diagramas sirven para revisar y como guía, pero la entrega pide StarUML.	alta
Informe técnico	**«Informe técnico con las decisiones de diseño y su justificación»** Las secciones 7 a 16 de esta guía sirven de base.	alta
Video	**«Video gameplay que muestre todas las funcionalidades implementadas»**	media
Mail al docente	Lo manda un solo integrante, con copia al resto, con el nombre del equipo y el apellido y nombre de cada uno.	media
Licencias y uso de IA	Anotar el origen y la licencia de cada imagen y sonido, y declarar el uso de IA en el informe.	media

PARA REVISAR EN EQUIPO

Son observaciones de la revisión del código integrado. Ninguna es grave; la idea es que todos sepan dónde están y decidan juntos.

1. Al copiar el ZIP del dominio, se borraron los comentarios de documentación de `Juego`, `EstadoJuego`, `ObservadorJuego`, `Direccion` y `Main`. Ya se recuperaron. Para la próxima, conviene traer los cambios con git (rama y pull request) y no copiando archivos.
2. `JuegoDePrueba` ya no se usa. Conviene borrar el paquete `sokoban.prueba` antes de entregar.
3. Mediator: solo el Jugador usa el mediador; la Caja no. Para la defensa, o la Caja pasa también por el mediador, o se explica por qué no (sección 11).
4. El deshacer guarda fotos del nivel. Se puede presentar como Memento si se nombra así en el informe (sección 12).
5. Cuando la caja frágil se rompe y el nivel queda sin solución, solo suena «derrota»: no hay cartel que avise que hay que reiniciar.
6. Detalles visuales: la caja llave tapa el dibujo del cerrojo y el jugador sobre hielo tapa el hielo.
7. Los 3 niveles son de prueba y muy chicos. Vale la pena diseñar niveles que muestren cada regla.
8. Los dos commits del dominio tienen el mismo mensaje («Added Player and boxes»), y `INTEGRACION_MEDIATOR_STRATEGY.md` habla de un ZIP. Conviene actualizar ese archivo o borrarlo.
9. El pull request del sonido sigue abierto: hay que aprobarlo en GitHub. `main` todavía no tiene la integración del arte (épocas) ni estas correcciones.

18 Ejercicio resuelto: ¿por qué el sonido es un Decorator y no un Observer?

> *Una pregunta muy probable en la defensa: las dos ideas sirven para «enterarse» de lo que pasa en el juego. Vale la pena saber por qué se eligió una.*

EJERCICIO RESUELTO Consigna: comparar dos formas de sumar el sonido sin tocar el dominio: (a) una clase que implemente `ObservadorJuego`, o (b) el `JuegoConSonido` que hay hoy.

Resolución:

- Con un **observador**, el sonido solo se entera cuando el dominio llama a `notificar()`, y eso pasa únicamente si algo cambió (`modelo/JuegoSokoban.java:45`: si el movimiento no se pudo, sale antes de notificar). Entonces nunca sabría que el jugador chocó contra una pared, ni que apretó deshacer sin que pasara nada.
- Con el **decorador**, el sonido ve el pedido en sí: sabe que se pidió mover, mira el estado antes y después, y si no cambió nada hace sonar el choque. Además distingue «iniciar nivel» de «reiniciar», que para un observador son el mismo aviso.
- Las dos opciones cumplen OCP (no tocan el dominio) y solo cambian `Main`. El decorador tiene más información, por eso se eligió.

```
ejemplo (no está en el repo): la opción (a) en Main
1   Juego juego = new JuegoSokoban();
2   juego.agregarObservador(new ReproductorSonido());   // (a) se entera solo de los cambios
3
4   // Lo que hay hoy (b), Main.java:19:
5   Juego juego = new JuegoConSonido(new JuegoSokoban()); // ve cada pedido, también los que
fallan
```

19 Autoevaluación

> *Tapá la columna de la derecha y respondé de memoria antes de mirar. Es la forma que más ayuda a llegar bien a la defensa.*

Pregunta	Respuesta
¿Por qué el dominio y el sonido entraron sin tocar la vista?	Porque la vista depende de la interfaz `Juego` y no de una clase concreta (DIP). Solo cambió `Main` (y se sumaron 5 símbolos en `CAPAS`).
¿Qué pasa, en orden, cuando apretás una flecha?	PantallaJuego → JuegoConSonido → JuegoSokoban (foto) → Jugador → MotorMovimiento → Caja y su estrategia → notificar a HUD y tablero → sonido.
En el Strategy, ¿quién es el contexto y quién elige la estrategia?	Contexto: `Caja`. La elige el constructor de `EstadoNivel` según la letra del .txt.
¿Qué hace la caja frágil y dónde está ese código?	Pierde 1 de resistencia por empuje; con 3 empujes se rompe y desaparece. `CajaFragilStrategy.despuesDeEmpujar`.
¿Qué ventaja da el Mediator y cuál es su punto débil acá?	Las reglas de movimiento están en un solo lugar y el Jugador no conoce cajas ni paredes. Punto débil: solo el Jugador le habla al mediador.
¿Cómo funciona el deshacer y cómo cumple 15 / 5 / 3?	Pila de fotos (máximo 15); cada uso saca 5 y vuelve a la última; un contador corta en 3 seguidos y vuelve a 0 al mover.
¿Cómo se calcula el puntaje?	max(0, 1000 - 10·movimientos - 5·empujes - 50·deshacer).
¿Por qué `JuegoConSonido` es un Decorator?	Implementa `Juego`, guarda otro `Juego` adentro, le delega todo y agrega el sonido.
¿Qué cambia entre `SonidoActivado` y `SonidoSilenciado`?	Qué hacen al reproducir (sonar o nada) y a cuál pasan con M. `Sonidos` no tiene ningún «if».
¿Cuáles son las tres piezas de un Singleton y en qué clases están?	Atributo `static` con la instancia, constructor privado y método `static` de acceso: en `Imagenes` y en `Sonidos`.
¿Por qué `EstadoJuego` es un `record` con `List.copyOf`?	Para que sea inmutable: ningún observador puede modificar lo que reciben los demás.

Pregunta	Respuesta
¿Dónde está el controlador del MVC?	La entrada (teclas y botones) está en `PantallaJuego` y `PanelHUD`; el controlador GRASP es `JuegoSokoban`.

Repasala en varias sesiones separadas (por ejemplo a los 1, 3, 7 y 14 días) en vez de todo junto la noche anterior.

20 Cómo mantener actualizados los diagramas

- Diagramas de clases: `python3 docs/uml/fuentes/diagramas.py` (las clases están en `clases.py` y la ubicación de cada caja y flecha en `diagramas.py`). Genera los SVG y los PNG.
- Diagramas de patrones: 3 Observer, 4 Facade, 5 Singleton, 8 Strategy, 9 Mediator, 10 Decorator, 11 State y 12 deshacer (Memento).
- Diagrama de secuencia: `docs/uml/fuentes/7-secuencia-mover.puml` (PlantUML).
- Esta guía: `node docs/guia/generar-pdf.js` a partir de `docs/guia/guia.html`.
- Cuando se agregue una clase, se suma en `clases.py` con el color de su autor y se ubica en el diagrama completo.

Esta guía y los diagramas se hicieron con asistencia de IA a partir del código del repositorio, y conviene declararlo en el informe técnico. El equipo es responsable de revisarlos y entenderlos.